

Architecture

AI Support Agents - Architectural Design Document

Version: 1.0

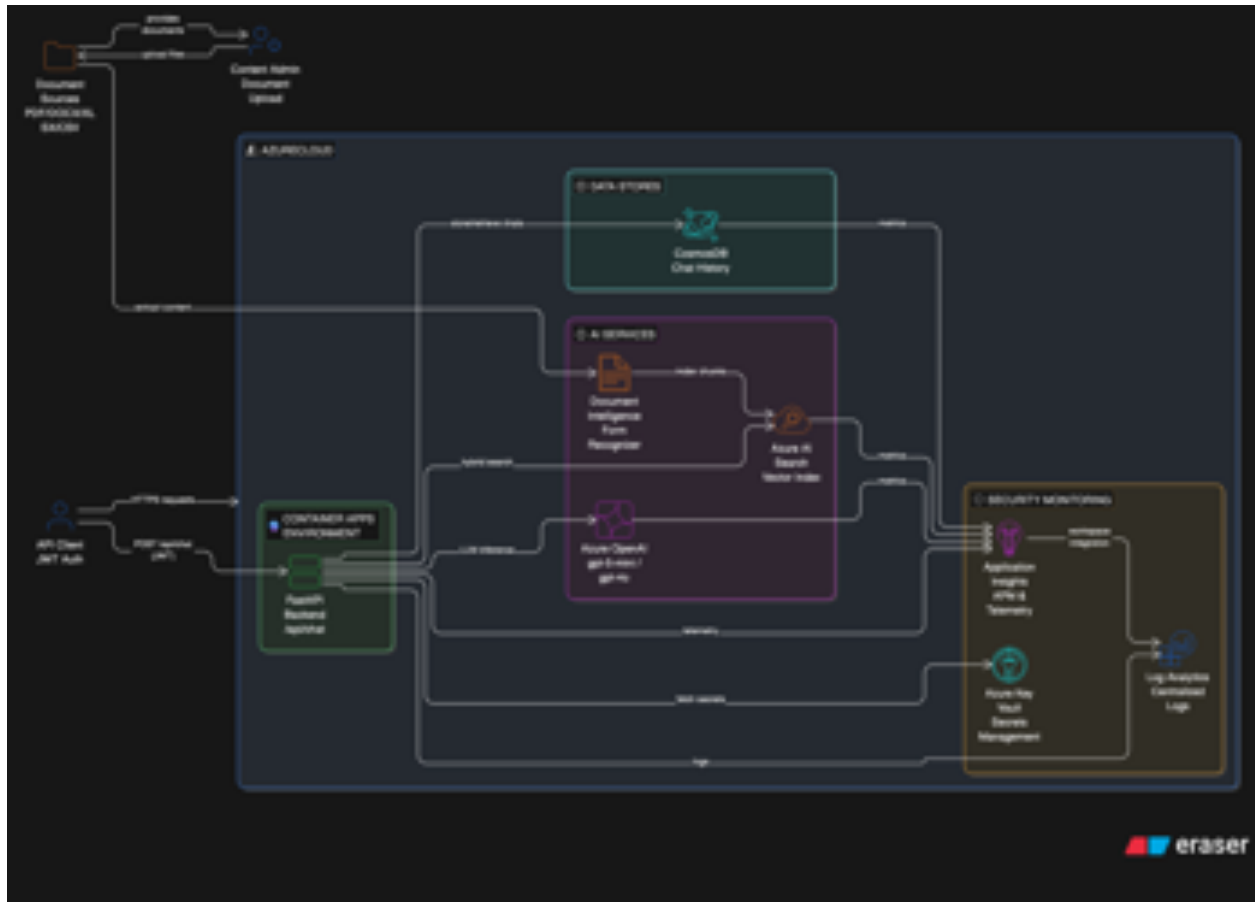
Last Updated: January, 16th 2026

Table of Contents

1. [High-Level Architecture](#)
 2. [End-to-End Data Flow Example](#)
 3. [Component Deep Dives](#)
 4. [Cross-Cutting Concerns](#)
 5. [Deployment Pipeline](#)
 6. [Monitoring & Observability](#)
-

1. High-Level Architecture

1.1 System Context Diagram



1.2 Request Lifecycle Walkthrough

1. **User Authentication:** Client authenticates and obtains JWT token (authentication handled externally)
2. **Query Submission:** Client sends POST to `/api/chat` with JWT in Authorization header.
3. **Routing Decision:** Backend's router LLM (`gpt-5-mini`) analyzes query and selects appropriate tool:
 - o `function_general_company`: For policy/document questions (uses RAG)
 - o `company_data_analysis`: For structured data queries (uses Pandas agent)
4. **Context Retrieval:** For RAG queries, Azure AI Search performs hybrid search (keyword + vector) to retrieve top-k relevant document chunks.
5. **Response Generation:** Retrieved context is injected into LLM prompt. Response streams token-by-token via Server-Sent Events (SSE).
6. **Persistence:** Conversation stored in CosmosDB for audit trail and feedback collection.

2. End-to-End Data Flow Example

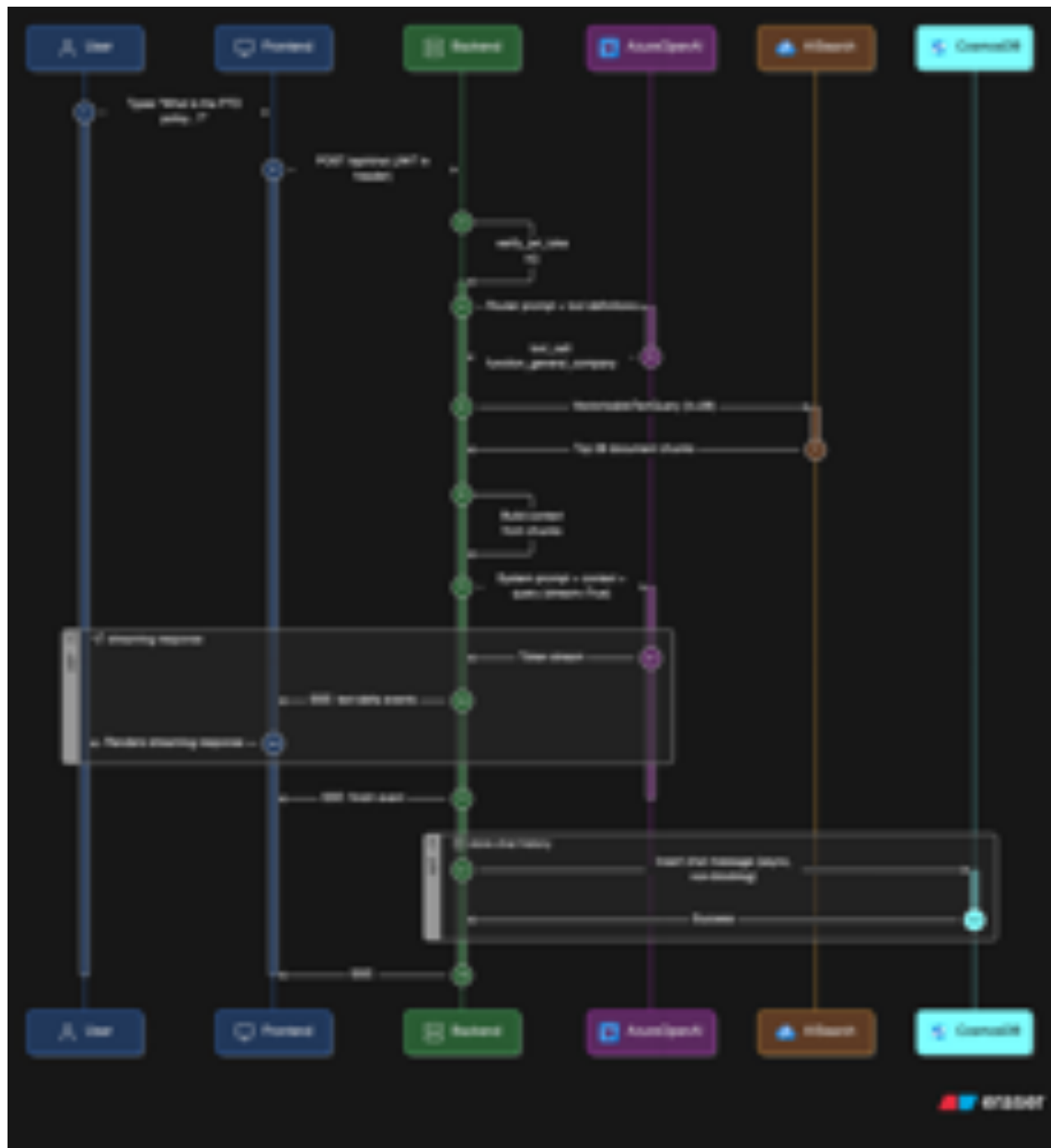
2.1 Sample Query Trace

User Query: "What is the PTO policy for employees with less than 5 years of service?"

2.2 Data Transformation at Each Hop

Stage	Component	Input	Transformation	Output
1	Frontend	User text input	Wraps in ClientMessage schema with id, role, content	<code>{"id": "msg-abc", "role": "user", "content": "What is..."}</code>
2	JWT Validation	HTTP Authorization header	Decodes JWT, validates signature via JWKS, checks expiry	<code>{"email": "john@lcs.com", "exp": 1737123456}</code>
3	Router LLM	User query + tool definitions	Selects tool via function calling	<code>{"tool": "function_general_company", "args": {"query": "..."}}</code>
4	Azure AI Search	Query text	Vectorizes query; performs hybrid search (BM25 + HNSW)	Top 20 chunks with scores, file_name, content, chapters
5	Context Builder	Search results	Concatenates chunks into context string	<code>"Title: HR_Policy.pdf\nContent: PTO accrual...\n\n..."</code>
6	Answer LLM	System prompt + context + query	Generates grounded response	Streaming tokens: "Employees", "with", " less", "..."
7	SSE Encoder	LLM tokens	Formats as SSE events	<code>data: {"type":"text-delta","delta":"Employees"}\n\n</code>
8	CosmosDB	Complete response + metadata	Creates document with partition key	<code>{"id": "msg-abc", "answer": "...", "toolsUsed": [...]}</code>

2.3 Sequence Diagram



2.4 Exact Payload Examples

Stage 1: Frontend Request

```

{
  "id": "chat-7f3a2b",
  "messages": [
    {
      "id": "msg-1",
    }
  ]
}

```

```

        "role": "user",
        "content": "What is the PTO policy for employees with less than 5 years
of service?"
    }
],
"top_k": 5,
"filters": null
}

```

Stage 4: Azure AI Search Response (truncated)

```

[
  {
    "file_name": "LCS_HR_Policy_Manual.pdf",
    "content": "## Paid Time Off (PTO)\n\nEmployees accrue PTO based on years
of service:\n- 0-4 years: 15 days annually\n- 5-9 years: 20 days
annually...",
    "content_summary": "This section describes PTO accrual rates based on
tenure...",
    "chapters": "Benefits > Paid Time Off",
    "@search.score": 0.892
  }
]

```

Stage 7: SSE Output Stream

```

data: {"type":"start","messageId":"msg-7f3a2b1c"}

data: {"type":"reasoning-start","id":"r1"}

data: {"type":"reasoning-delta","id":"r1","delta":"Searching company
documents..."}

data: {"type":"reasoning-end","id":"r1"}

data: {"type":"text-start","id":"text-1"}

data: {"type":"text-delta","id":"text-1","delta":"According to"}

data: {"type":"text-delta","id":"text-1","delta":" the LCS HR Policy
Manual,"}

data: {"type":"text-delta","id":"text-1","delta":" employees with less than 5
years"}

...

data: {"type":"text-end","id":"text-1"}

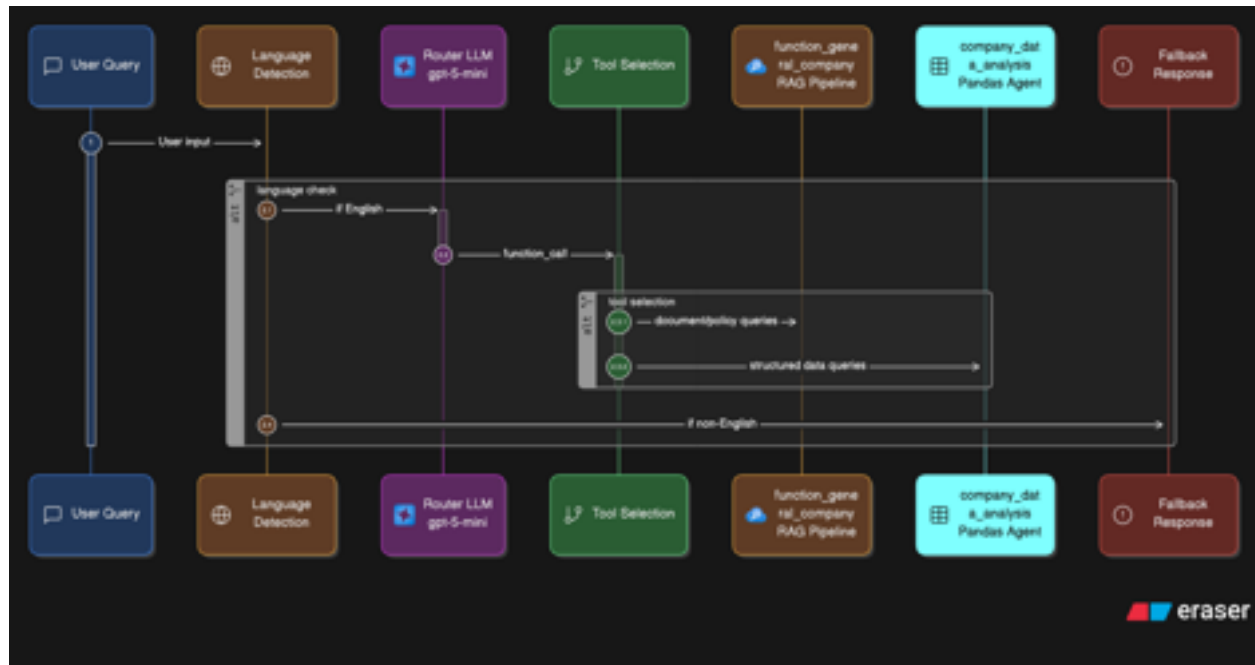
data:
{"type":"finish","messageMetadata":{"finishReason":"stop","usage":{"promptTok
ens":1842,"completionTokens":156}}}

data: [DONE]

```

3. Component Deep Dives

3.1 Query Router (stream.py)



Purpose: Routes user queries to the appropriate specialized agent based on query intent.

Implementation Details:

1. **Language Detection:** Performed by the router LLM itself via prompt engineering. Non-English queries receive a fixed fallback response.
2. **Tool Selection Logic** (pseudocode):
 - **Company Data Analysis:** Selected when the query references specific identifiers (e.g., person names, community IDs, system IDs) or requests structured data operations.
 - **General Company Function:** Selected for inquiries regarding policies, guidelines, or internal documents. This serves as the default fallback for ambiguous queries.
3. **Streaming Protocol:** Uses `AsyncAzureOpenAI.chat.completions.create(stream=True)` with custom SSE formatting.

Failure Handling:

- **No Tool Selection:** In rare cases where the LLM fails to select a tool, the system falls back to a direct response.

- **Execution Exceptions:** If a selected tool throws an exception, the error is caught, logged with a traceback, and returned to the user via a friendly SSE error message.
- **Non-English Input:** Non-English queries are intercepted immediately (via Router LLM system prompt), preventing further unnecessary LLM inference costs.

External Libraries:

Library	Version	Why Chosen
openai	1.x	Official SDK with native async streaming support; type-safe; actively maintained
httpx	0.x	Required for custom timeout configuration; trust_env=False prevents proxy interference

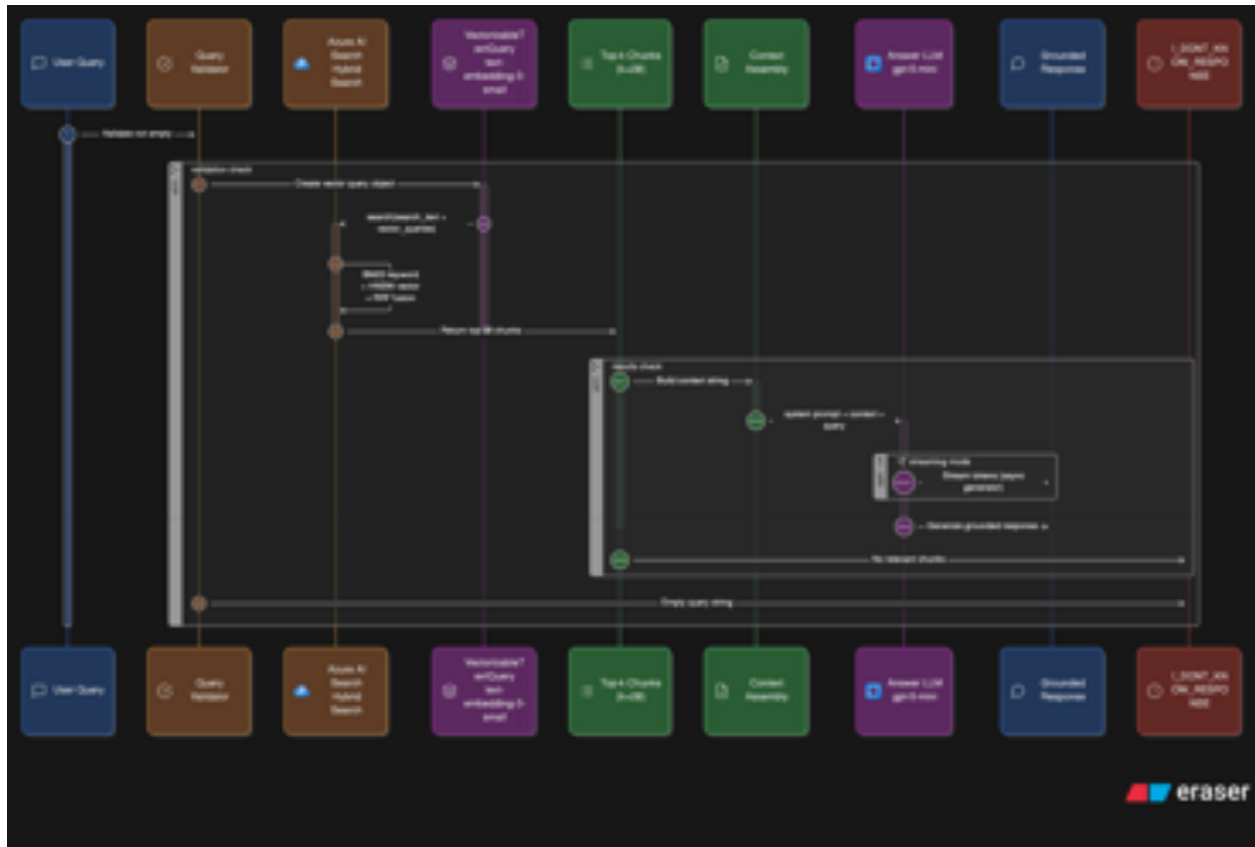
Trade-offs:

- **Intent Detection:** We chose an LLM-based router over keyword regex matching. While regex is faster, it is brittle and fails on paraphrased queries. The LLM handles ambiguity and variations in phrasing with significantly higher accuracy.
- **Model Selection (gpt-5-mini):** The classification task is relatively simple compared to generation. Using the "mini" model offers a 10x cost reduction and lower latency (~200ms) compared to gpt-4o, without sacrificing routing accuracy.

Configuration Rationale:

Parameter	Value	Why
timeout_keep_alive	300s	LLM responses can take 30-60s for complex queries; 5-minute keep-alive prevents client disconnect
httpx.Timeout(read=30.0)	30s read	Balance between allowing slow responses and failing fast on stalled connections
max_retries=0	No retries	Streaming responses cannot be retried mid-stream; let client handle retry

3.2 RAG Agent (utils/general/agent.py)



Purpose: Retrieves relevant document chunks and generates grounded answers using RAG.

1. Prerequisites

Index Schema:

The search index uses the following schema to store document chunks:

```
{
  "name": "lcs_chatbot",
  "fields": [
    {"name": "chunk_id", "type": "Edm.String", "key": true},
    {"name": "file_name", "type": "Edm.String", "filterable": true},
    {"name": "specific_chunk_identifier", "type": "Edm.String", "searchable": true},
    {"name": "chapters", "type": "Edm.String", "searchable": true},
    {"name": "content_summary", "type": "Edm.String", "searchable": true},
    {"name": "content", "type": "Edm.String", "searchable": true},
    {"name": "content_vector", "type": "Collection(Edm.Single)", "dimensions": 1536}
  ],
  "vectorSearch": {
    "algorithms": [{"name": "myHnsw", "kind": "hnsw"}],
    "profiles": [{"name": "myHnswProfile", "algorithm": "myHnsw", "vectorizer": "myOpenAI"}],
  }
}
```



```

    "vectorizers": [{"name": "myOpenAI", "kind": "azureOpenAI", "model":
"text-embedding-3-small"}]
  }
}

```

Key fields:

- `content_vector`: 1536-dim embeddings from text-embedding-3-small
- `content`: Original text content for BM25 search
- `file_name`: Source document for citations
- `chapters`: Document structure for context

Embedding Model Choice:

Ranked by MTEB (English) benchmark from [OpenAI's official comparison](#):

1. **text-embedding-3-large**: 64.6% MTEB, \$0.13/1M tokens, 3072 dimensions
2. **text-embedding-3-small**: 62.3% MTEB, \$0.02/1M tokens, 1536 dimensions (Selected)
3. **text-embedding-ada-002**: 61.0% MTEB, \$0.10/1M tokens, 1536 dimensions

We utilize `text-embedding-3-small` for vector generation. Based on MTEB benchmarks, this model delivers 90% of the performance of larger models at roughly 15% of the cost. Given that the corpus is primarily English-language HR documentation, this trade-off provides optimal value without sacrificing retrieval accuracy.

2. Search Implementation

Hybrid Search:

Combines BM25 keyword matching with HNSW vector similarity. Azure AI Search handles score fusion internally using Reciprocal Rank Fusion (RRF).

```

vector_query = VectorizableTextQuery(
    text=query, fields="content_vector", k_nearest_neighbors=top_k
)
results = search_client.search(
    search_text=query,
    vector_queries=[vector_query],
    top=top_k # top_k = 20
)
results = list(results)

```

BM25 (Best Matching 25): Keyword ranking algorithm that scores docs by term frequency, adjusts for document length, and weights rare terms higher via IDF. Triggered by `search_text=query` parameter.

HNSW (Hierarchical Navigable Small World): Graph-based algorithm for fast approximate nearest neighbor search in vector spaces. Handles millions of vectors with >95% recall. Triggered by `vector_queries=[vector_query]` parameter.

Score Fusion/Reciprocal Rank Fusion (RRF): Combines rankings from BM25 and HNSW into unified results. No tuning required.

How RRF works:

1. **Parallel execution:** BM25 + HNSW run simultaneously
2. **Reciprocal ranking:** $\text{Score} = 1/(\text{rank} + 60)$ where $k=60$ is optimal
3. **Score summation:** Sum scores across methods
4. **Re-ranking:** Sort by combined score

Example:

BM25 Results:	HNSW Results:
1. Doc A (1/61)	1. Doc B (1/61)
2. Doc B (1/62)	2. Doc A (1/62)
3. Doc C (1/63)	3. Doc D (1/63)

Final RRF Scores:

Doc A:	$1/61 + 1/62 = 0.0325$	(rank 1)
Doc B:	$1/62 + 1/61 = 0.0325$	(rank 2)
Doc C:	$1/63$	$= 0.0159$ (rank 3)
Doc D:	$1/63$	$= 0.0159$ (rank 4)

Real example: Query "vacation policy" → BM25 finds docs with "vacation", HNSW finds docs about time-off/PTO, RRF surfaces docs with both concepts.

References:

- [Azure AI Search - Hybrid Search Ranking \(RRF\)](#)
- [HNSW Original Paper \(2016\)](#)

Query Embedding: Uses `VectorizableTextQuery` which delegates embedding to Azure AI Search's configured vectorizer (text-embedding-3-small). This avoids a separate embedding API call.

Reasoning: No tuning needed, handles different score scales, emphasizes consensus across methods. Hybrid search (keyword + vector) rather than pure vector search because keyword search catches exact matches like policy names and acronyms that embeddings might miss.

3. Response Generation

Retrieved chunks are concatenated into a context window and passed to the Answer LLM (gpt-5-mini). The system enforces a strict **Grounding Prompt** that explicitly forbids the model from

using outside knowledge. If the retrieved context does not contain the answer, the model is instructed to return a standard fallback message ("I don't know") rather than hallucinating.

Context Assembly:

```
context = "\n\n".join(
    f"Title: {r.get('file_name', '')}\nContent: {r.get('content', '')}"
    for r in results
)
```

Grounding Prompt (from prompts.py):

You are a helpful assistant.

IMPORTANT RULES:

- Answer the user's question **only** using the information explicitly provided in the given context.
- The provided context is the **only** source of truth.
- Do **NOT** use prior knowledge, general knowledge, assumptions, inference, or reasoning beyond what is stated in the context.
- Do **NOT** guess, infer missing details, or fill in gaps.
- Do **NOT** hallucinate or fabricate any information.
- Do **NOT** provide partial answers, explanations, or summaries not supported by the context.
- If the provided context contains **no relevant information**, ignore the user query itself and respond exactly with the fallback message: {I_DONT_KNOW_RESPONSE}

External Libraries:

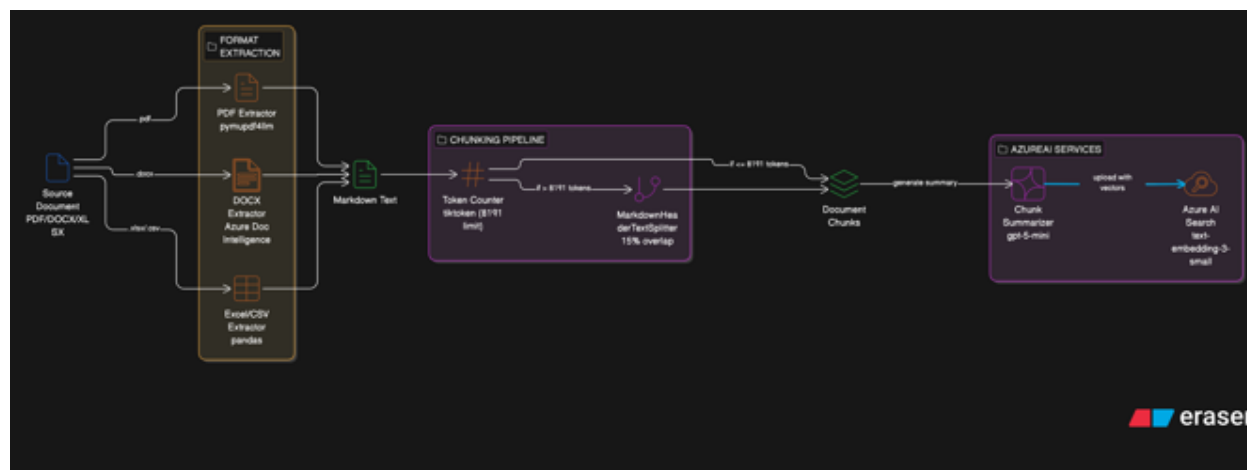
Library	Version	Why Chosen
azure-search-documents	11.x	Official Azure SDK; handles retry logic and connection pooling
openai	1.x	Async streaming for token-by-token generation

Configuration Rationale:

Parameter	Value	Why
top_k	20	The k=20 retrieval size balances recall vs context dilution - k=5 risks missing relevant chunks while k=50 exceeds the context window. 20 chunks occupy ~8k tokens, leaving room for system prompt and response (max_completion_tokens=3000)
max_completion_tokens	3000	Longest expected answer is ~2000 tokens; 3000 provides headroom without risking runaway generation

k_nearest_neighbors	20	Matches top_k; HNSW returns this many candidates before fusion with BM25
---------------------	----	--

3.3 Document Chunker (RAG-utils/chunker.py)



Purpose: Converts source documents to indexed, searchable chunks with semantic summaries.

Implementation Details:

1. Format Detection & Extraction:

- **PDF Documents:** Processed using `pymupdf4llm` to convert content to Markdown. This library was selected for its ability to preserve document structure and table formatting accurately.
- **DOCX Files:** Handled by **Azure Document Intelligence** using the prebuilt-layout model. This approach ensures high accuracy for complex formatting, such as multi-column layouts and embedded images.
- **Legacy DOC Files:** Converted via a two-stage process: `LibreOffice` converts the file to PDF, which is then processed by `pymupdf4llm`.
- **Spreadsheets (XLSX/XLS/CSV):** Converted to Markdown tables using `pandas`. This ensures that tabular data remains structured and readable within the LLM context.

2. Chunking Strategy:

To maintain semantic coherence, the chunking strategy prioritizes document structure over arbitrary token limits:

- **Primary Splitting:** Uses `MarkdownHeaderTextSplitter` to divide content based on markdown headers (H1–H23). This respects the document's logical hierarchy.
- **Secondary Splitting:** If a chunk exceeds the 8191-token limit, it is further split using token-based logic with a **15% overlap** to preserve context across boundaries.
- **Table Handling:** Large tables are split row-wise, ensuring that the header row is preserved in every resulting chunk to maintain column context.

3. Summarization:

Before indexing, each chunk is passed through an LLM to generate a semantic summary. This summary helps the retrieval system understand the "gist" of the chunk, which is particularly useful for sections that rely heavily on implicit context.

4.

```
prompt = "Summarize the following text into 5-10 sentences..."
summary = oai_client.chat.completions.create(...)
```

5. Index Schema:

The Azure AI Search index is structured to support both vector and keyword search:

- o `chunk_id` (*String*): Unique UUID acting as the primary key.
- o `file_name` (*String*): The source document name, enabled for filtering.
- o `content` (*String*): The full text of the chunk, used for full-text search.
- o `content_summary` (*String*): LLM-generated summary, used to augment retrieval relevance.
- o `content_vector` (*Collection*): 1536-dimensional embedding generated by `text-embedding-3-small`.
- o `chapters` (*String*): A hierarchical path string (e.g., "HR Policy > PTO > Accrual") derived from headers.

External Libraries:

Library	Version	Why Chosen
<code>pymupdf4llm</code>	0.x	Best-in-class PDF extraction; preserves tables and structure; MIT license
<code>tiktoken</code>	0.x	OpenAI's official tokenizer; accurate token counts for context window management
<code>langchain-text-splitters</code>	0.x	Robust markdown-aware splitting; handles nested headers
<code>azure-ai-documentintelligence</code>	1.x	Highest accuracy for complex DOCX with images/tables comparing to other open-source alternatives.

Trade-offs:

- **PDF Extraction:** We prioritized `pymupdf4llm` over Azure Document Intelligence for PDFs to reduce operational costs and latency, as the open-source library provides sufficient accuracy for standard text documents.
- **Token Limit (8191):** The chunk size limit is aligned with the maximum input size of the `text-embedding-3-small` model. Smaller chunks (e.g., 4096) were rejected to avoid unnecessary fragmentation of long sections, means that the Language Model might lose the "big picture".

Example: a large "IT Security Policy" document that is **6,000 tokens** long.

- **Scenario A (Limit = 4096):** The document is forced to split into **Chunk 1** (Part A) and **Chunk 2** (Part B).
 - *Risk:* If the user asks "Summarize the IT security protocols," the search might only retrieve Chunk 1. The AI misses half the rules because they are in Chunk 2, which wasn't retrieved.
- **Scenario B (Limit = 8191 - Current Strategy):** The entire 6,000-token document fits into **one single chunk**.
 - *Result:* The embedding vector represents the *entire* policy. When retrieved, the LLM reads the full context from start to finish, no rules are missed.
- **Overlap (15%):** A 15% overlap was chosen to ensure continuity. Zero overlap risks losing context at boundaries, while higher values (e.g., 30%) introduce excessive redundancy.

Example: "...to reset the server, press the red button. WARNING: Do not press it if the light is flashing."

- **Scenario A (0% Overlap):**
 - **Chunk 1 (ends):** "...to reset the server, press the red button. WARNING: Do not press it"
 - **Chunk 2 (starts):** "if the light is flashing."
 - *Failure:* If a user asks "When should I NOT press the red button?", neither chunk has the answer. Chunk 1 has the warning but not the condition. Chunk 2 has the condition but not the warning.
- **Scenario B (15% Overlap):**
 - **Chunk 1:** Ends at token 8191.
 - **Chunk 2:** Starts back at token 6963, repeating the last section of Chunk 1.
 - **Content in Chunk 2:** "...to reset the server, press the red button. WARNING: Do not press it if the light is flashing." ... (continues with new text).
 - *Success:* Chunk 2 now contains the **complete logical statement**. The AI can read the full warning in one go.
- **Summarization Strategy:** We utilize generative LLM summarization rather than extractive methods because summaries capture semantic meaning and implicit information that keyword-based extraction might miss.

Configuration Rationale:

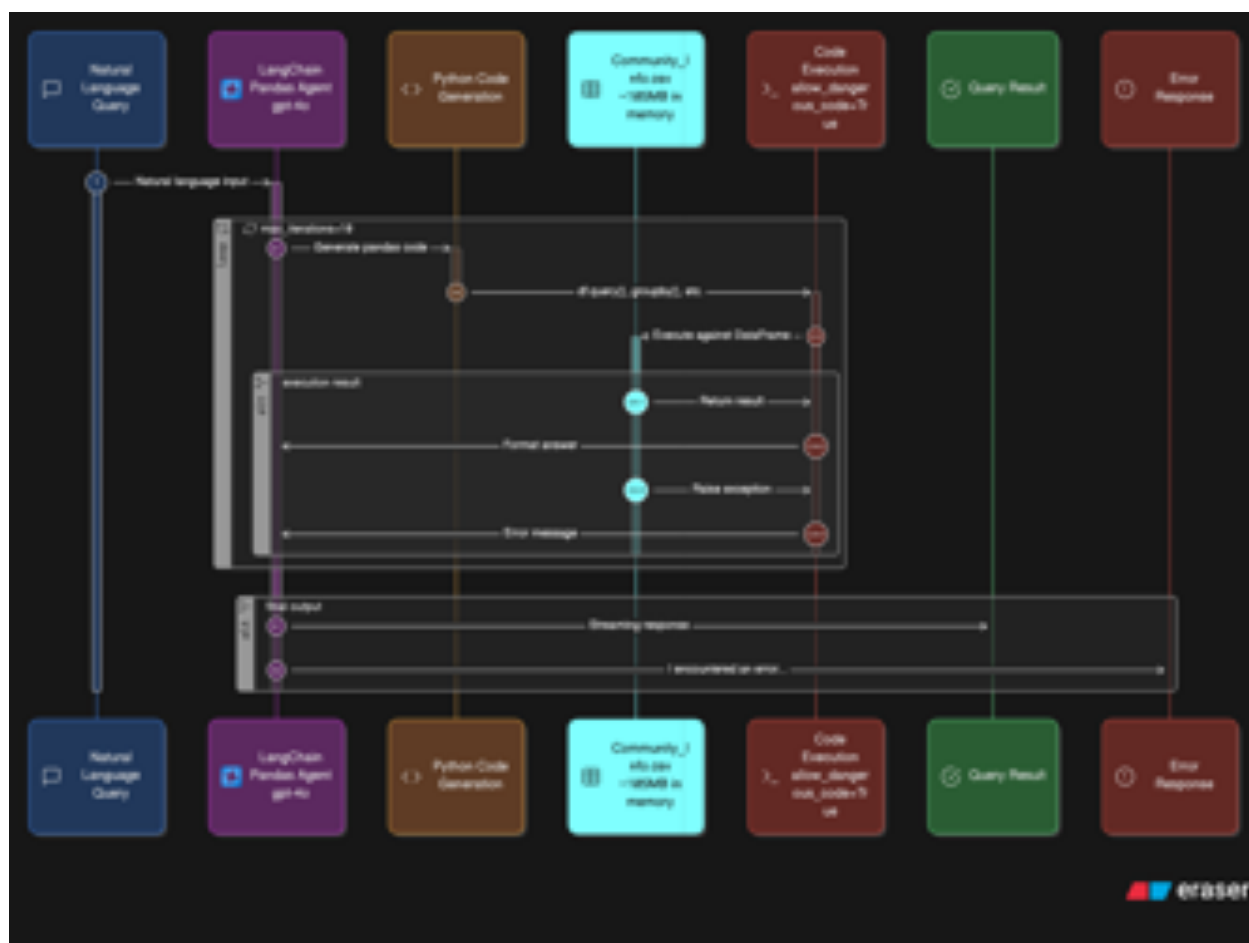
Parameter	Value	Why
TOKEN_LIMIT	8191	Maximum input for text-embedding-3-small model

overlap_ratio	0.15 (15%)	Ensures cross-chunk context continuity without excessive duplication
max_tokens (summary)	3000	Summaries should be concise; 3000 allows for complex table descriptions

Edge Cases & Failure Modes:

Scenario	Behavior	Side Effect
Document extraction fails	Logs error; skips document to prevent pipeline blockage. Admin notification is required for resolution	Missing document from index; admin notification needed
Single row exceeds token limit	Row included as-is without splitting	This may result in embedding truncation but preserves data integrity.
LibreOffice not installed (DOC)	Processing legacy DOC files requires LibreOffice. If absent, the subprocess will raise a <code>RuntimeError</code>	DOC files cannot be processed; requires manual conversion

3.4 CSV Data Agent (`utils/csv/agent.py`)



The CSV Data Agent provides capabilities for structured data analysis by dynamically generating and executing Python `pandas` code against tabular datasets. This allows the system to answer quantitative questions (e.g., "How many employees are in the West Division?") that cannot be accurately addressed by vector search.

Implementation Details:

1. **DataFrame Loading:** `Community_Info.csv` (~105MB) loaded at module import time. Cached in memory for fast queries. Eliminates database latency, ensuring instantaneous access for downstream query operations.
 2. **Agent Configuration:**
- ```

3. pandas_agent = create_pandas_dataframe_agent(
4. llm=llm_4o,
5. df=df_community,
6. agent_type="tool-calling",
7. allow_dangerous_code=True,
8. number_of_head_rows=5,
9. max_iterations=10,
10.)

```



- `allow_dangerous_code=True` is enabled to permit the execution of generated Python code within the container.
  - `number_of_head_rows=5` is provided to the LLM to give it schema visibility without consuming excessive tokens.
10. **Streaming:** Uses LangChain's `astream_events(version="v2")` to capture and yield token-level output from the final response generation step.
11. **Agent Caching:** Agents are cached per settings configuration to avoid recreation overhead.

### External Libraries:

| Library                | Version | Why Chosen                                                              |
|------------------------|---------|-------------------------------------------------------------------------|
| langchain-experimental | 0.x     | Provides <code>create_pandas_dataframe_agent</code> with code execution |
| langchain-openai       | 0.x     | AzureChatOpenAI integration with streaming support                      |
| pandas                 | 2.x     | Industry-standard DataFrame library; required by agent                  |

### Trade-offs:

| Decision                               | Alternative Considered | Why Rejected                                                                                            |
|----------------------------------------|------------------------|---------------------------------------------------------------------------------------------------------|
| gpt-4o for code generation             | gpt-5-mini             | Complex pandas operations require stronger reasoning; 4o is 3x better at code generation                |
| <code>allow_dangerous_code=True</code> | Sandboxed execution    | Sandboxing limits pandas functionality; data is read-only, no file system access; code is LLM-generated |
| In-memory DataFrame                    | External Database      | 105MB fits in memory; eliminates network latency; simplifies architecture                               |
| Single DataFrame                       | Multiple DataFrames    | Current scope only uses <code>Community_Info</code> ; extensible to multiple later                      |

### Configuration Rationale:

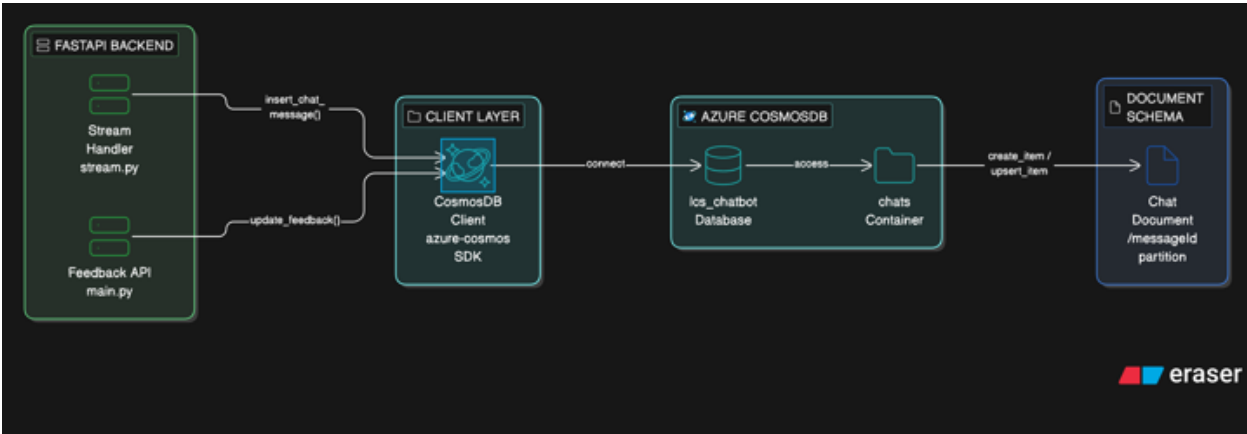
| Parameter                   | Value | Why                                                                         |
|-----------------------------|-------|-----------------------------------------------------------------------------|
| <code>temperature</code>    | 0     | Deterministic code generation; avoids hallucinated function names           |
| <code>max_iterations</code> | 10    | Prevents infinite loops on complex queries; most complete in 2-3 iterations |

|                     |   |                                                      |
|---------------------|---|------------------------------------------------------|
| number_of_head_rows | 5 | Gives LLM schema context without overwhelming prompt |
|---------------------|---|------------------------------------------------------|

Edge Cases & Failure Modes:

| Scenario                             | Behavior                                                                                                                                                               | Side Effect                                                                                                                                                                   |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Query causes infinite loop           | Stops at 10 iterations                                                                                                                                                 | Returns partial result or error to prevent resource exhaustion.                                                                                                               |
| Generated code throws exception      | Exceptions raised by the Pandas engine (e.g., dividing by zero)<br><br>The error message is fed back to the LLM, which attempts to self-correct in the next iteration. | If loop threshold is exceeded, user sees "I encountered an error..."                                                                                                          |
| Query asks about non-existent column | Agent attempts query; fails                                                                                                                                            | The tool returns the error message as the output containing <code>KeyError</code> to inform the agent of the valid schema, allowing it to explain the limitation to the user. |

3.5 Conversation Storage (utils/cosmosdb.py)



**Purpose:** Persists chat conversations for audit, feedback collection, and analytics.

Implementation Details:

1. **Document Schema:**
2. {
3.   "\_\_id": "msg-abc123",
4.   "messageId": "msg-abc123",

```

5. "conversationHistory": [...],
6. "answer": "According to the policy...",
7. "feedback": "up" | "down" | null,
8. "userEmail": "john@lcs.com",
9. "timestamp": "2026-01-16T16:07:00Z",
10. "finishReason": "stop",
11. "toolsUsed": ["function_general_company"]
12. }

```

13. **Partition Strategy:** Partitioned by `messageId`. Optimizes for the most frequent access pattern: retrieving or updating a specific message (e.g., adding user feedback) by its unique identifier.

14. **Async Operations:** Uses `asyncio.to_thread()` to wrap synchronous CosmosDB SDK calls, to prevent blocking the main FastAPI event loop

### External Libraries:

| Library      | Version | Why Chosen                                                                   |
|--------------|---------|------------------------------------------------------------------------------|
| azure-cosmos | 4.x     | Official Azure SDK; handles retries, connection pooling, conflict resolution |

### Trade-offs:

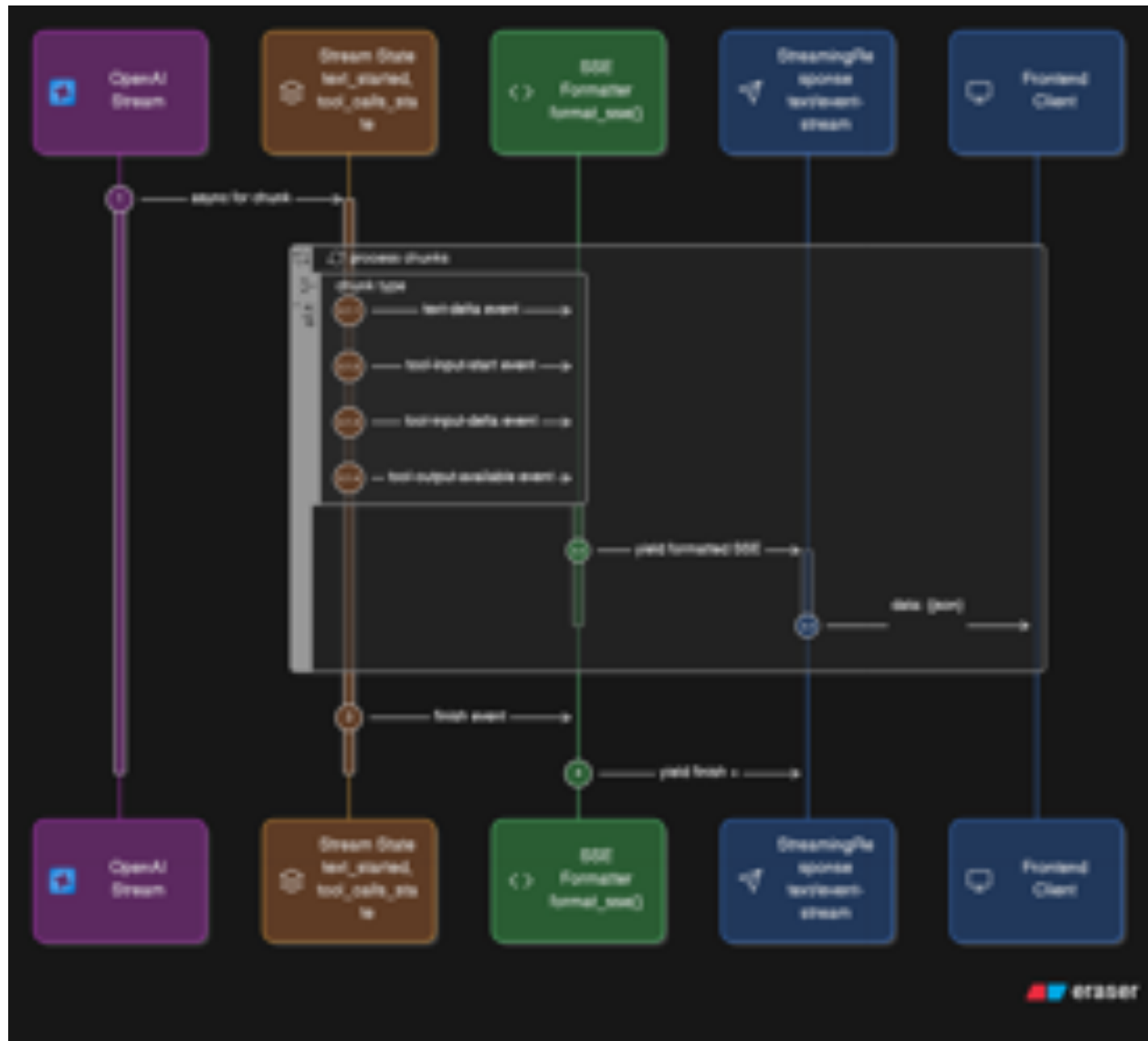
| Decision                                   | Alternative Considered                             | Why Rejected                                                                               |
|--------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------|
| CosmosDB Serverless                        | Provisioned throughput                             | Unpredictable workload; serverless auto-scales; no capacity planning needed                |
| <code>messageId</code> as partition key    | <code>conversationId</code> or <code>userId</code> | Most queries are single-message lookups; <code>messageId</code> provides best distribution |
| Sync SDK wrapped in <code>to_thread</code> | Async SDK                                          | Async CosmosDB SDK is less mature; <code>to_thread</code> is simple and reliable           |

### Edge Cases & Failure Modes:

| Scenario                    | Behavior                                             | Side Effect                                 |
|-----------------------------|------------------------------------------------------|---------------------------------------------|
| CosmosDB write fails        | Logs error; does not fail user request               | Message lost from audit trail; non-blocking |
| Concurrent feedback updates | <code>CosmosAccessConditionFailedError</code> raised | Rare; last-write-wins semantics acceptable  |

|                                |                                                                                                                                                                            |                               |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Message not found for feedback | If a feedback request targets a non-existent message ID, the operation returns <code>False</code> and logs a warning, preventing system crashes due to stale client state. | Frontend may show stale state |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|

### 3.6 Streaming Response Handler (`stream.py`)



**Purpose:** Transforms OpenAI streaming responses into SSE events for client consumption.

#### Implementation Details:

##### 1. Event Types Emitted:

- Lifecycle Events:** `start` (stream initialized), `finish` (stream complete with usage stats).

- **Reasoning Events:** reasoning-start, reasoning-delta, reasoning-end (for communicating internal thought processes).
- **Content Events:** text-start, text-delta, text-end (for standard chat responses).
- **Tooling Events:** tool-input-start, tool-input-delta, tool-output-available (for visualizing agent actions).

## 2. State Tracking:

The handler maintains an internal state machine (`tool_calls_state`) to track partial tool calls. This allows it to aggregate fragmented JSON arguments streamed by the LLM into complete, valid objects before execution. The `tool_calls_state` dictionary acts as a buffer/accumulator, holding partial fragments in memory until the LLM signals completion.

When the LLM decides to invoke a tool (e.g., search documents, query data), it doesn't send the function arguments as a single complete JSON object. Instead, it streams them token-by-token, resulting in broken fragments like `{"que, ry": "PT, O policy"}`.

The state tracker is used for:

- **Fragment Assembly:** Accumulates these incomplete tokens into a complete, valid JSON object (e.g., `{"query": "PTO policy"}`). Only when the full command is received does the backend parse the JSON and execute the corresponding tool function.
- **Real-Time Feedback:** While buffering, it emits incremental updates (`tool-input-delta` events) to the frontend. This allows the UI to display the agent's "thinking" process in real-time (e.g., showing "Searching for: PTO policy..." as it types), providing immediate visual feedback before the tool actually runs.

## 3. HTTP Headers:

```
4. headers={
5. "Cache-Control": "no-cache, no-transform",
6. "Content-Encoding": "none",
7. "Connection": "keep-alive",
8. }
```

- `Cache-Control: no-cache, no-transform`: Prevents CDN/proxy buffering which breaks SSE
- `Content-Encoding: none` to disable gzip compression, each data packet is sent immediately.
- `Connection: keep-alive`: Maintains the persistent connection required for SSE.

## Design Decisions & Trade-offs

- **Custom SSE Protocol:** Provides fine-grained control over event types (`tool-input-delta`, `reasoning-start`, etc.) to support real-time UI feedback that off-the-shelf libraries don't offer.

- **Single Stream:** Multiplexes all events (text, tool calls, results) over one HTTP connection instead of separate WebSockets, simplifying client logic.
- **Inline Execution:** Executes tools synchronously within the stream generator to eliminate polling delays and maintain event ordering.

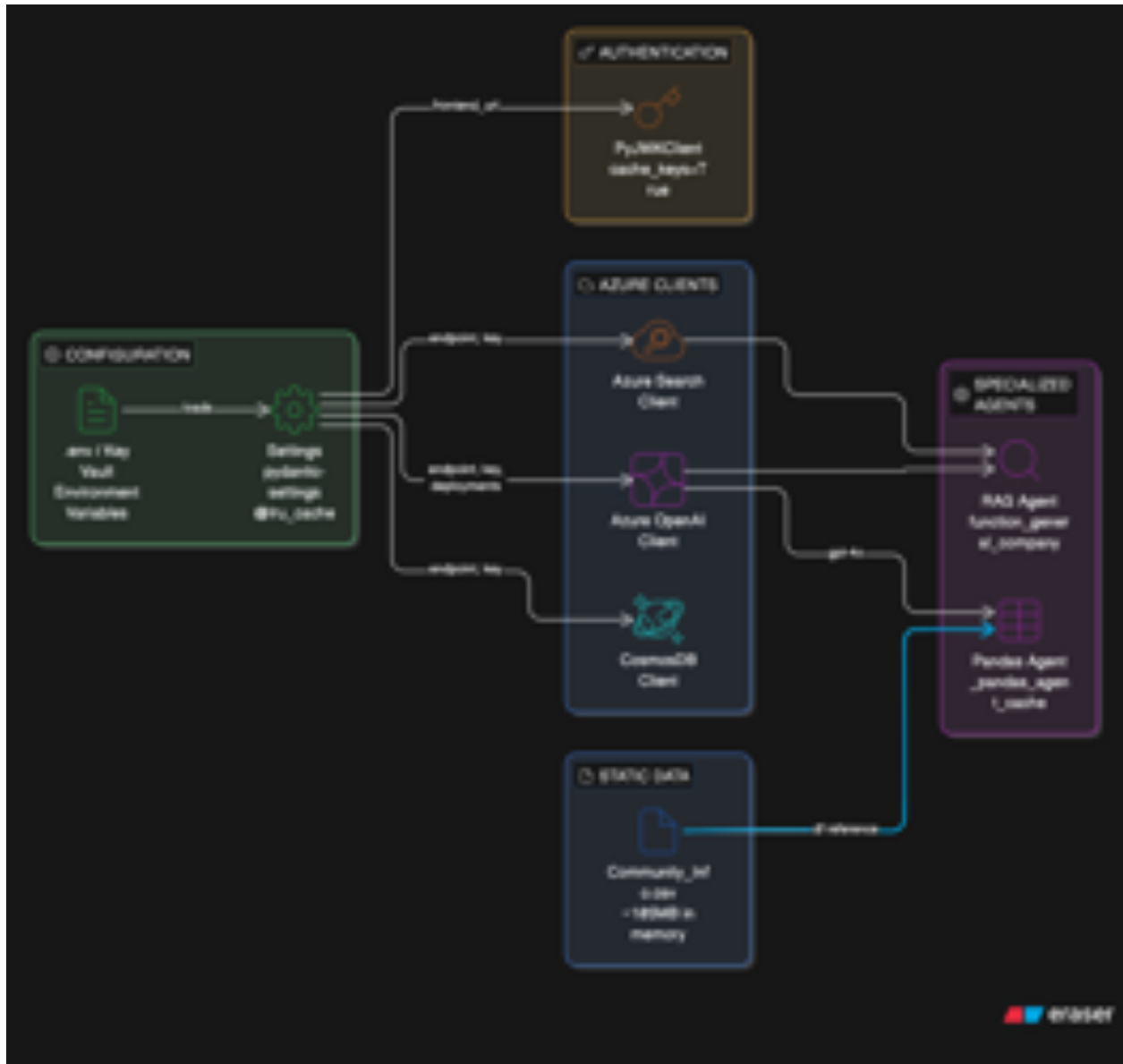
**Configuration Rationale:**

| Parameter        | Value                  | Why                                           |
|------------------|------------------------|-----------------------------------------------|
| Cache-Control    | no-cache, no-transform | Prevents CDN/proxy buffering which breaks SSE |
| Content-Encoding | none                   | Disables gzip which would buffer the stream   |

---

## 4. Cross-Cutting Concerns

### 4.1 Shared State & Dependencies



### Shared Resources:

| Resource           | Scope         | Cache Strategy                              | Risk if Unavailable    |
|--------------------|---------------|---------------------------------------------|------------------------|
| Settings           | App lifetime  | @lru_cache on <code>get_settings()</code>   | App fails to start     |
| PyJWKClient        | App lifetime  | <code>cache_keys=True</code> internal cache | Auth fails; 403 errors |
| Community_Info.csv | Module import | In-memory DataFrame                         | CSV agent fails        |

|                     |              |                                   |                                   |
|---------------------|--------------|-----------------------------------|-----------------------------------|
| _pandas_agent_cache | App lifetime | Dict keyed by endpoint+deployment | Recreates agent (slow first call) |
|---------------------|--------------|-----------------------------------|-----------------------------------|

### Coupling Analysis:

| Component A      | Component B        | Coupling Type       | If A Changes...                                  |
|------------------|--------------------|---------------------|--------------------------------------------------|
| stream.py        | prompts.py         | Import dependency   | Prompt changes affect routing behavior           |
| stream.py        | general/agent.py   | Function call       | Agent signature changes require stream.py update |
| general/agent.py | prompts.py         | Import dependency   | System prompt changes affect answer quality      |
| csv/agent.py     | Community_Info.csv | File dependency     | CSV schema changes may break queries             |
| main.py          | utils/utils.py     | Settings dependency | Env var changes require restart                  |

## 4.2 Bottlenecks & Scaling Limits

| Component              | Bottleneck                   | Symptom                            | Mitigation                                    |
|------------------------|------------------------------|------------------------------------|-----------------------------------------------|
| <b>Azure OpenAI</b>    | Token-per-minute (TPM) quota | 429 errors; increased latency      | Increase quota; implement exponential backoff |
| <b>CSV Agent</b>       | 105MB DataFrame in memory    | High memory usage; slow cold start | Pre-warm containers; consider chunked loading |
| <b>Azure AI Search</b> | Query units (QU) per second  | Increased latency                  | Upgrade SKU from free to basic/standard       |
| <b>CosmosDB</b>        | RU consumption spikes        | 429 throttling                     | Serverless auto-scales; monitor RU usage      |
| <b>Container Apps</b>  | CPU/memory limits            | Container restarts; slow responses | Increase limits; scale replicas               |

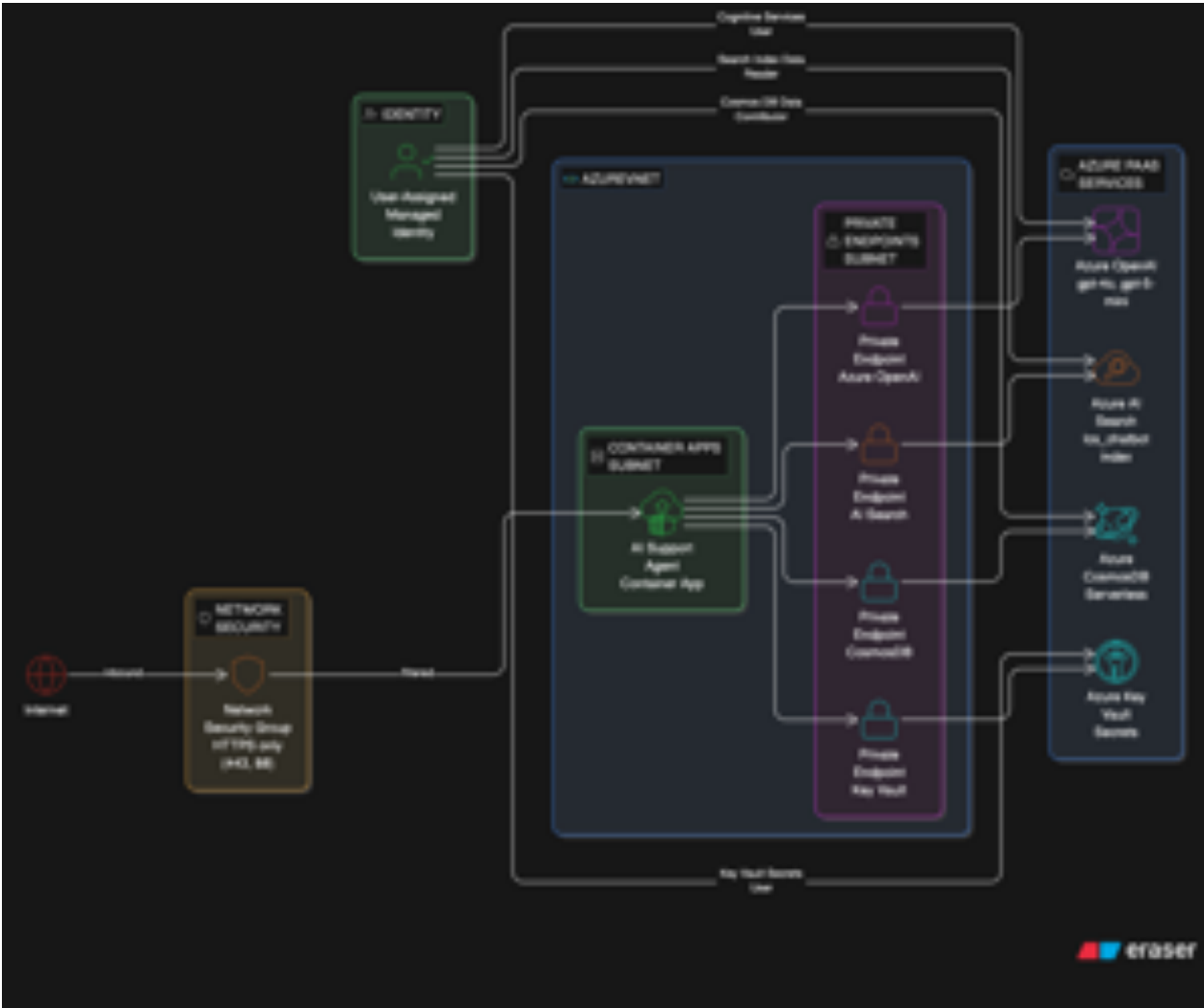
### Load Characteristics:

| Metric             | Expected Load | Breaking Point        | Monitoring           |
|--------------------|---------------|-----------------------|----------------------|
| Concurrent users   | 50-100        | 500+ (OpenAI quota)   | Application Insights |
| Queries per minute | 30-60         | 200+ (depends on TPM) | OpenAI metrics       |



|                        |           |                          |                   |
|------------------------|-----------|--------------------------|-------------------|
| Document uploads       | 10-20/day | 1000+/day (indexing lag) | AI Search metrics |
| Response latency (p95) | < 10s     | > 30s (timeout)          | Custom metrics    |

### 4.3 Security Architecture



#### Authentication & Authorization:

| Layer                     | Mechanism        | Configuration                        |
|---------------------------|------------------|--------------------------------------|
| Frontend -> Backend       | JWT Bearer token | EdDSA algorithm; JWKS validation     |
| Backend -> Azure Services | API Keys         | Keys stored in environment variables |
| Secrets                   | Key Vault        | Credentials stored in Key Vault      |

#### Network Security (Production):

| Component       | Access Control                | Private Endpoint       |
|-----------------|-------------------------------|------------------------|
| Azure OpenAI    | VNet rules + Private Endpoint | Yes                    |
| Azure AI Search | VNet rules + Private Endpoint | Yes (if not free tier) |
| CosmosDB        | VNet rules + Private Endpoint | No (not configured)    |

#### 4.4 Observability Stack

| Tool                 | Purpose                  | Data Captured                            |
|----------------------|--------------------------|------------------------------------------|
| Application Insights | APM, distributed tracing | Request traces, dependencies, exceptions |
| Log Analytics        | Centralized logging      | Container logs, Azure service logs       |
| Azure Monitor        | Metrics & alerts         | CPU, memory, request rates, error rates  |
| CosmosDB             | Audit trail              | All conversations, feedback, tool usage  |

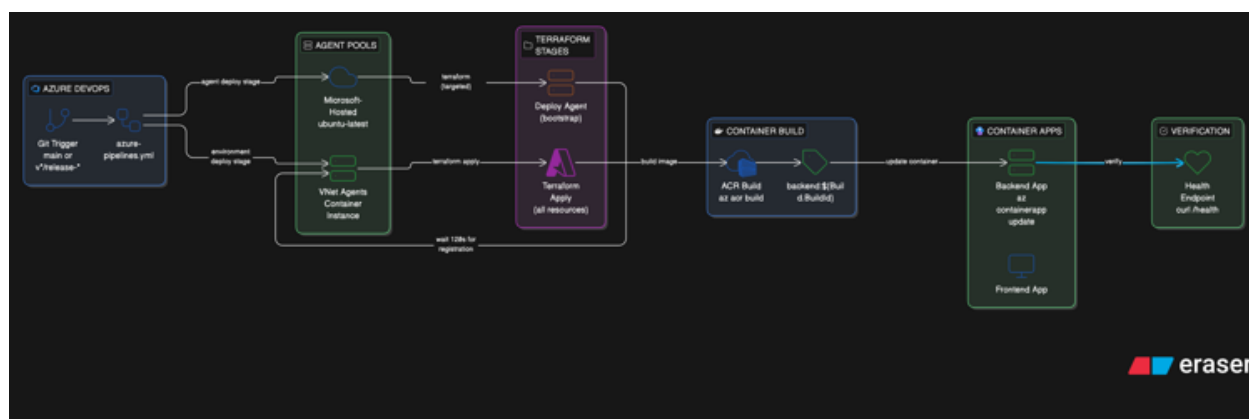
#### 4.5 Failure Modes Summary

| Failure                | Detection            | Impact                  | Recovery                             |
|------------------------|----------------------|-------------------------|--------------------------------------|
| OpenAI service down    | HTTP 5xx             | All queries fail        | Retry with backoff; alert on-call    |
| AI Search unavailable  | HTTP 5xx             | RAG queries fail        | CSV agent still works; alert on-call |
| CosmosDB write failure | Exception logged     | Conversations not saved | Non-blocking; data loss              |
| JWT validation failure | 403 response         | User cannot access      | User re-authenticates                |
| Container crash        | Health probe failure | Temporary outage        | Auto-restart by Container Apps       |
| Memory exhaustion      | OOMKilled            | Container restart       | Increase memory limit                |

---

## 5. Deployment Pipeline

### 5.1 CI/CD Architecture



## Pipeline Structure:

| Stage            | Agent Pool       | Purpose                           | Trigger              |
|------------------|------------------|-----------------------------------|----------------------|
| DeployAgent_Dev  | Microsoft-hosted | Bootstrap self-hosted agent infra | main branch          |
| Deploy_Dev       | vnet-agents      | Full Terraform + container deploy | After agent ready    |
| DeployAgent_Prod | Microsoft-hosted | Bootstrap prod agent infra        | v* or release-* tags |
| Deploy_Prod      | vnet-agents-prod | Full Terraform + container deploy | After agent ready    |

## Why Self-Hosted Agents:

- Private endpoints require VNet access for Terraform to reach Azure PaaS services
- Microsoft-hosted agents cannot access resources behind private endpoints
- Self-hosted agent runs as Azure Container Instance within the VNet

## 5.2 Deployment Sequence



## Terraform Targeting Strategy:

The pipeline uses a two-phase Terraform approach to solve the chicken-and-egg problem:

| Phase          | Runs On            | Targets                         | Resources                                      |
|----------------|--------------------|---------------------------------|------------------------------------------------|
| <b>Phase 1</b> | Microsoft-hosted   | - target=azurerm_*devops_agent* | Subnet, NSG, NAT Gateway, Container Instance   |
| <b>Phase 2</b> | Self-hosted (VNet) | Full state                      | All infrastructure including private endpoints |

## Container Build Process:

```
az acr build \
 --registry $ACR_NAME \
 --image backend:${Build.BuildId} \
 --image backend:latest \
 --file Dockerfile .
```

## 5.3 Environment Promotion

| Environment | Trigger             | State Backend            | Agent Pool       |
|-------------|---------------------|--------------------------|------------------|
| <b>Dev</b>  | Push to main        | lcsdevaiterraformstorage | vnet-agents      |
| <b>Prod</b> | Tag v* or release-* | lcsprodterraformstorage  | vnet-agents-prod |

## Deployment Safeguards:

| Safeguard         | Implementation                                                 |
|-------------------|----------------------------------------------------------------|
| State locking     | Azure Storage blob lease (auto-cleanup on failure)             |
| Plan before apply | <code>terraform plan -out=file.tfplan</code> saved as artifact |
| Format check      | <code>terraform fmt -check -recursive</code> (non-blocking)    |
| Validation        | <code>terraform validate</code> before plan                    |
| Rollback          | Revert tag + re-run pipeline (immutable container images)      |

---

## 6. Monitoring & Observability

### 6.1 Observability Architecture



## 6.2 Diagnostic Settings Configuration

| Resource              | Log Categories                                                                                                     | Metrics    | Retention |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|------------|-----------|
| Azure OpenAI          | Audit, RequestResponse                                                                                             | AllMetrics | 30 days   |
| AI Search             | OperationLogs                                                                                                      | AllMetrics | 30 days   |
| CosmosDB              | DataPlaneRequests, QueryRuntimeStatistics, PartitionKeyStatistics, PartitionKeyRUConsumption, ControlPlaneRequests | Requests   | 30 days   |
| Key Vault             | AuditEvent, AzurePolicyEvaluationDetails                                                                           | AllMetrics | 30 days   |
| Document Intelligence | Audit, RequestResponse                                                                                             | AllMetrics | 30 days   |

## 6.3 Application Insights Integration

### Backend Instrumentation:

| Component           | Telemetry Type      | Data Captured            |
|---------------------|---------------------|--------------------------|
| FastAPI requests    | Request traces      | Duration, status, URL    |
| OpenAI calls        | Dependency traces   | Model, tokens, latency   |
| AI Search queries   | Dependency traces   | Query text, result count |
| CosmosDB operations | Dependency traces   | Operation type           |
| Exceptions          | Exception telemetry | Stack trace, context     |

Note: Application Insights integration is configured in the infrastructure but specific instrumentation needs to be added to the backend code.

## 6.4 Key Metrics & Alerts

| Metric                  | Source            | Warning Threshold | Critical Threshold |
|-------------------------|-------------------|-------------------|--------------------|
| Response latency (p95)  | App Insights      | > 5s              | > 10s              |
| Error rate (5xx)        | App Insights      | > 1%              | > 5%               |
| OpenAI token usage      | OpenAI diagnostic | > 80% quota       | > 95% quota        |
| CosmosDB RU consumption | CosmosDB metrics  | > 70% provisioned | > 90% provisioned  |

|                       |                |               |               |
|-----------------------|----------------|---------------|---------------|
| Container CPU         | Container Apps | > 70%         | > 90%         |
| Container memory      | Container Apps | > 70%         | > 90%         |
| Health probe failures | Container Apps | 2 consecutive | 5 consecutive |

## 6.5 Log Queries (KQL)

### Request Performance:

```
requests
| where timestamp > ago(1h)
| summarize
 avg(duration),
 percentile(duration, 95),
 percentile(duration, 99)
 by bin(timestamp, 5m)
| render timechart
```

### OpenAI Usage by Model:

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.COGNITIVESERVICES"
| where Category == "RequestResponse"
| extend model = tostring(parse_json(properties_s).model)
| summarize count(), sum(toint(properties_s.usage_total_tokens)) by model,
bin(TimeGenerated, 1h)
```

### Failed Requests Analysis:

```
requests
| where success == false
| summarize count() by resultCode, name
| order by count_desc
```

## 6.6 Audit Trail (CosmosDB)

All conversations are persisted to CosmosDB for audit and analytics:

| Field     | Purpose                                 |
|-----------|-----------------------------------------|
| messageId | Unique identifier (partition key)       |
| userId    | Authenticated user ID                   |
| query     | User's original question                |
| response  | Full assistant response                 |
| tool_used | Agent invoked (General, Data, Combined) |



|           |                                          |
|-----------|------------------------------------------|
| feedback  | User feedback (thumbs up/down, comments) |
| timestamp | ISO 8601 timestamp                       |

**Audit Query Example:**

```
SELECT c.userId, c.tool_used, c.timestamp
FROM c
WHERE c.timestamp > @startDate
ORDER BY c.timestamp DESC
```

## Appendix A: Environment Variables

| Variable                         | Description                                 | Required |
|----------------------------------|---------------------------------------------|----------|
| FRONTEND_URL                     | Base URL for JWKS endpoint (JWT validation) | Yes      |
| AZURE_OPENAI_ENDPOINT            | Azure OpenAI service endpoint               | Yes      |
| AZURE_OPENAI_KEY                 | Azure OpenAI API key                        | Yes      |
| AZURE_OPENAI_VERSION             | API version (default: 2024-12-01-preview)   | No       |
| AZURE_OPENAI_DEPLOYMENT_GPT_MINI | Deployment name for routing LLM             | Yes      |
| AZURE_OPENAI_DEPLOYMENT_GPT_4O   | Deployment name for data analysis           | Yes      |
| AZURE_SEARCH_ENDPOINT            | Azure AI Search endpoint                    | Yes      |
| AZURE_SEARCH_INDEX_NAME          | Index name (default: lcs_chatbot)           | No       |
| AZURE_SEARCH_KEY                 | Azure AI Search admin key                   | Yes      |
| AZURE_COSMOSDB_ENDPOINT          | CosmosDB account endpoint                   | Yes      |
| AZURE_COSMOSDB_KEY               | CosmosDB primary key                        | Yes      |
| AZURE_COSMOSDB_DATABASE_NAME     | Database name                               | Yes      |
| AZURE_COSMOSDB_CONTAINER_NAME    | Container name                              | Yes      |

## Appendix B: Acceptance Criteria Mapping

| Requirement | Implementation | Validation |
|-------------|----------------|------------|
|-------------|----------------|------------|

|                              |                                                  |                                               |
|------------------------------|--------------------------------------------------|-----------------------------------------------|
| Answer accuracy >= 95%       | Hybrid search; k=20 retrieval; LLM summarization | Ground truth test suite (eventually DeepEval) |
| English only                 | Router LLM language detection                    | Non-English fallback response                 |
| JWT validation               | JWT validation in backend                        | -                                             |
| Support PDF, DOCX, XLSX, CSV | Format-specific extractors in chunker            | -                                             |
| Streaming responses          | SSE protocol                                     | -                                             |